Week 4 Report: Logging Process for the PIC16F72 Simulator

Project-Based Learning – Microcontroller Simulator | Name/Team: [your name] | Date: [date]

1. Objective

Build a standalone logging process that receives messages from the simulator over a local TCP socket and writes them to one timestamped log file, giving a single place to debug instruction execution.

2. Design

Item	Choice
Transport	TCP socket on 127.0.0.1:45773 (local only)
Framing	One connection = one message; connection close marks the end
Output	logs/simulator.log, format: time | level | message
Shutdown	Message **SHUTDOWN** stops the logger

3. Implementation (Python, standard library only)

```
import logging, os, socket

HOST, PORT = "127.0.0.1", 45773
os.makedirs("logs", exist_ok=True)

logging.basicConfig(
    filename="logs/simulator.log", level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s",
    encoding="utf-8")
logger = logging.getLogger("PIC16F72")

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen(10)
    print(f"Logging process listening on {HOST}:{PORT}", flush=True)

    running = True
    while running:
        conn, _ = server.accept()
        with conn:
            data = b""
            while True:
                chunk = conn.recv(4096)
                if not chunk:
                    break
                data += chunk

        message = data.decode("utf-8", errors="replace").strip()
        if message:
            logger.info(message)
        if message == "SHUTDOWN":
            running = False

print("Logging process stopped.", flush=True)
```

- **SO_REUSEADDR** allows an immediate restart without a port error.
- **listen(10)** queues up to 10 pending connections.
- **recv(4096)** in a loop handles messages longer than one packet.
- **errors="replace"** prevents bad bytes from crashing the logger.

4. Problems Found and Fixed

- **Indentation errors:** loops and `with` blocks were not nested, causing an IndentationError.
- **Socket closed early:** `accept()` ran after the server socket's `with` block ended; it now runs inside it.
- **Logging outside the loop:** decode and log now run once per connection.
- **Shutdown:** `break` was not tied to the outer loop; replaced with a `running` flag.

5. Testing

Test	Input	Expected	Actual
Normal message	CPU: executed MOVLW 0x55	One timestamped line	[fill in]
Multiple messages	3 messages in a row	3 lines, in order	[fill in]
Empty message	empty string	Nothing written	[fill in]
Long message	over 4096 bytes	Full message logged	[fill in]
Shutdown	SHUTDOWN	Logger exits cleanly	[fill in]

Add a terminal screenshot and a snippet of simulator.log here.

6. Limitations

- Handles one connection at a time (threading or selectors could be added).
- All messages are logged at INFO level; no separate warning/error levels yet.
- Any local program that knows the port can send SHUTDOWN.
- Log folder depends on the directory the script is launched from.

7. Learning Outcomes

- Inter-process communication with sockets.
- Configuring Python's logging module (file handler, format).
- How indentation and scope affect resource lifetime.

8. Conclusion

The logger now runs as a separate component, records timestamped messages from a local socket, and shuts down cleanly on command. It will support debugging for the rest of the simulator.